

It can be applied as follows:

```
qc = QuantumCircuit(2)
qc.cx(0,1)
qc.draw(output='mpl')
```

Notice that we now use two qubits (q0 and q1) to invoke a quantum circuit, and cx gate takes two parameters (0,1). This means that the gate is controlled on q0 (control qubit) and is applied on q1 (target bit). It works according to the state value of q0; if q0 = |0>, then there will be no change in q1's state, but if q0 is in state |1>, then q1's state will be flipped. This control can be changed in the following way:

```
qc = QuantumCircuit(2)
qc.cx(1,0)
qc.draw(output='mpl')
```

Now, the cx gate is applied at q0 (target qubit) with control on q1 (control qubit). The state of q0 will be flipped if q1 is in state |1>; otherwise, it will be left as it is. Both the resultant gates are displayed in Figure 6.

b. CZ gate: Similar to the CX gate,

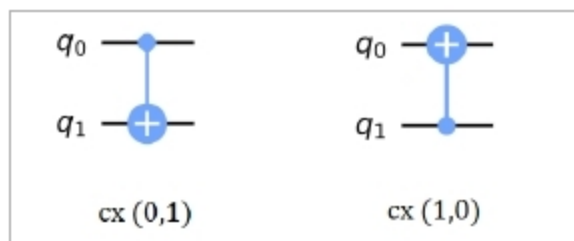


Figure 6: CNOT gate (i) controlled on q0; and (ii) controlled on q1

we have the CZ (Controlled Z) gate. Its job is to apply phase flip to the target qubit according to the state of control qubit. The matrix for this gate is as follows:

$$CZ = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & -1 \end{bmatrix}$$

We will visualise two cases for this gate.
Case-1: When input is 00>:

```
qc = QuantumCircuit(2)
qc.cz(0,1)
qc.draw(output='mpl')
```

To visualise *statevectors* of both

qubits separately on Bloch spheres, the following code is used:

```
backend = Aer.get_backend("statevector_simulator")
result = execute(qc, backend).result().get_statevector()
print(result)
plot_bloch_multivector(result)
```

Result: [1.+0.j 0.+0.j 0.+0.j
-0.+0.j]

This is a good point to introduce the Q-sphere. Till now we have used Bloch spheres to visualise the single qubit circuits. Bloch spheres are specifically meant to visualise single-qubit *statevectors* only, while Q-spheres are used to visualise combined states of multi-qubit circuits:

```
from qiskit.visualization import plot_state_qsphere
from qiskit.quantum_info import Statevector
sv = Statevector.from_label('00').
evolve(qc)
plot_state_qsphere(sv.data, show_state_phases=True)
```

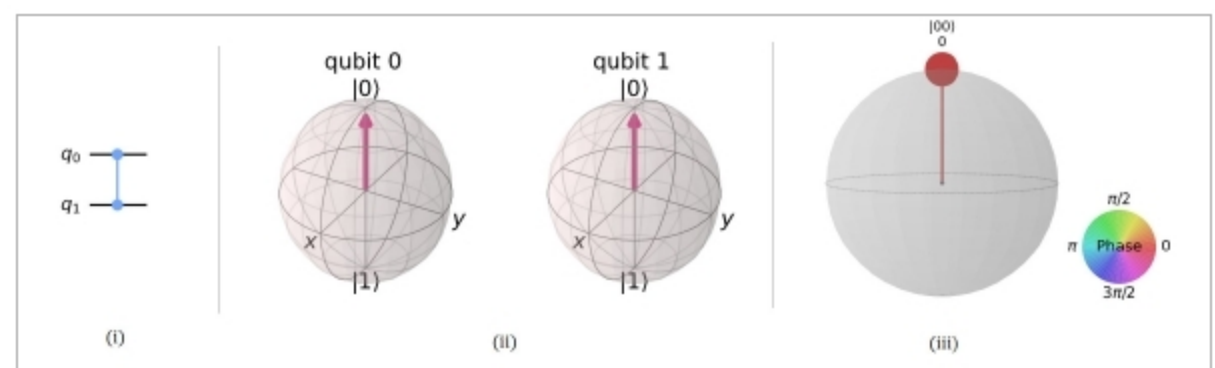


Figure 7: (i) the cz gate in a 2-qubit circuit; (ii) separate *statevectors* plotted on Bloch spheres; (iii) combined state of the circuit plotted on the Q-sphere

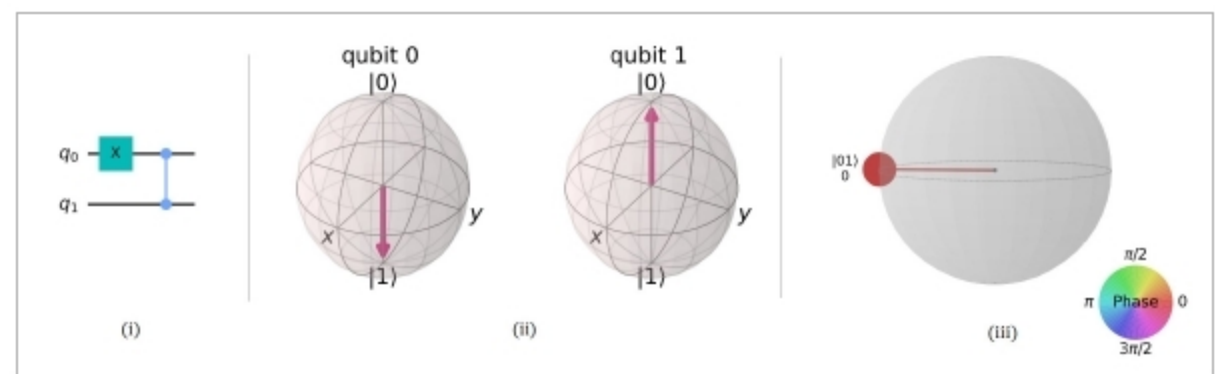


Figure 8: (i) The cz gate applied on |1>; (ii) Bloch spheres displaying both qubit states; and (iii) Q-sphere showing the combined state of the circuit

Case-2: When the control qubit (q0) is in state |1>:

```
qc = QuantumCircuit(2)
qc.x(0)
qc.cz(0,1)
qc.draw(output='mpl')
```

This circuit can be executed in the same way as case 1. Figure 8 shows the output.

statevector result is: [0.+0.j
1.+0.j; 0.+0.j -0.+0.j]; Z gate has applied a phase on the target qubit.

Almost all gates can be used in a controlled way in Qiskit.

3. Three-qubit gates

There are gates that take three qubits to be applied and operate according to their states, like the ones described below

a. CCNOT (Toffoli) gate: The CCNOT or Toffoli gate is a three-qubit gate that is controlled on two qubits and has one target qubit. It flips the target qubit based on the